Online Payments

Easypaisa – RSA 2048 Encryption Guide (Initiate MA API)

Integration Guide

Table of Contents

1. Easypaisa Payment Solution 3
2. RSA Encryption 4
   2.1 Public/Private Key Exchange 4
   2.2 Signature Generation 5
   2.3 Signature Verification 6

1. Easypaisa Payment Solution

Easypaisa is an electronic payment solution that easily and seamlessly integrates with any online website and/or shopping cart and enables users to pay online through Payment Gateway. Easypaisa is built upon the latest state of the art technologies and toolset that leverages many features a secure payment solution system must have. The users require no special technology or business license; only a valid email address to have an account on Easypaisa.

2. RSA Encryption

In order to use the web-services of OPS, it is necessary for the client to send a digital signature along with each API request. This signature is used by Easypaisa to authenticate the client before processing the request. The steps required to perform this encryption are described below:

Client and Easypaisa must exchange RSA keys before making API calls, and the length of RSA key must be 2048 bits. When making API call to Easypaisa, client uses the RSA private key to sign the API request. After receiving the API request, Easypaisa will use the client's RSA public key to verify whether the signature is matched to the content of API request. Similarly, when client receives the API response, it is highly recommended that client verifies the signature of API response by using Easypaisa's RSA public key.

2.1 Public/Private Key Exchange

An RSA key pair contains the private key and the public key. The private key is required for generating the signature, while the public key is used for verifying the signature. Many tools can be used to generate the RSA key pair. After the RSA2 key pair is generated, you must exchange the public key with the Easypaisa server for signature verification by completing the following steps:

Upload your public key in Easypaisa merchant portal.

Log in to the portal and visit the following path: "Account Settings -> Public Key Configuration -> Upload Public Key". Upload your public key here.

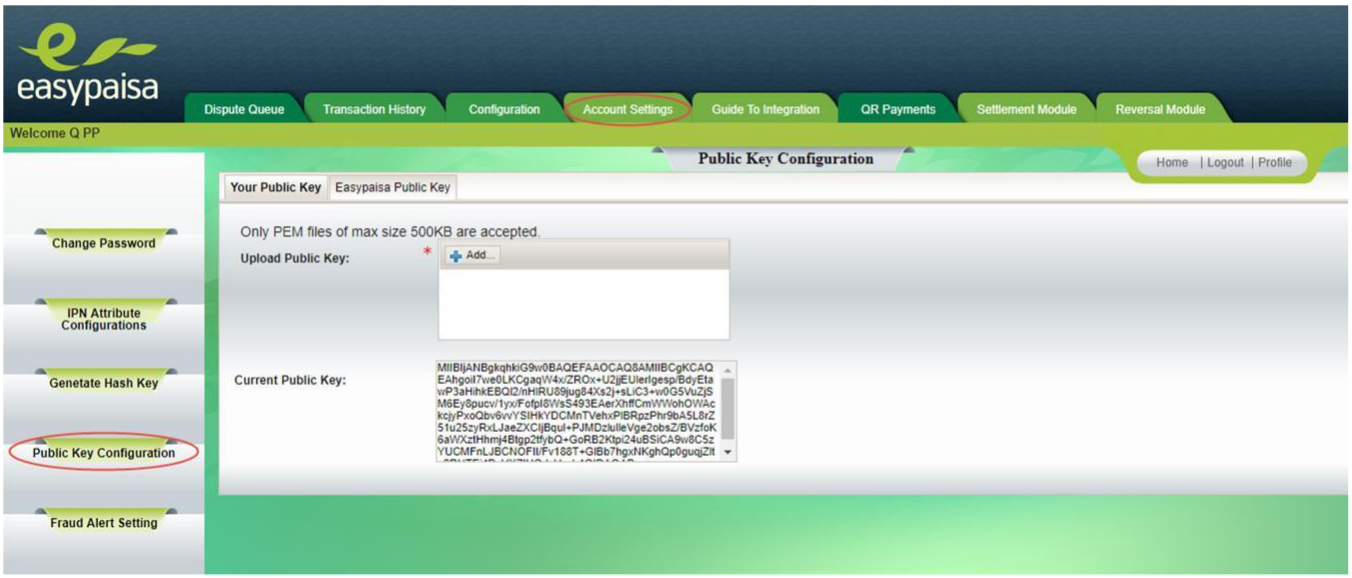

Figure 1 – Public Key Configuration (Upload)

2. **Download Easypaisa public key. [OPTIONAL]**

On the same path, enter the "Easypaisa Public Key" tab and click on the download button to download Easypaisa public key.

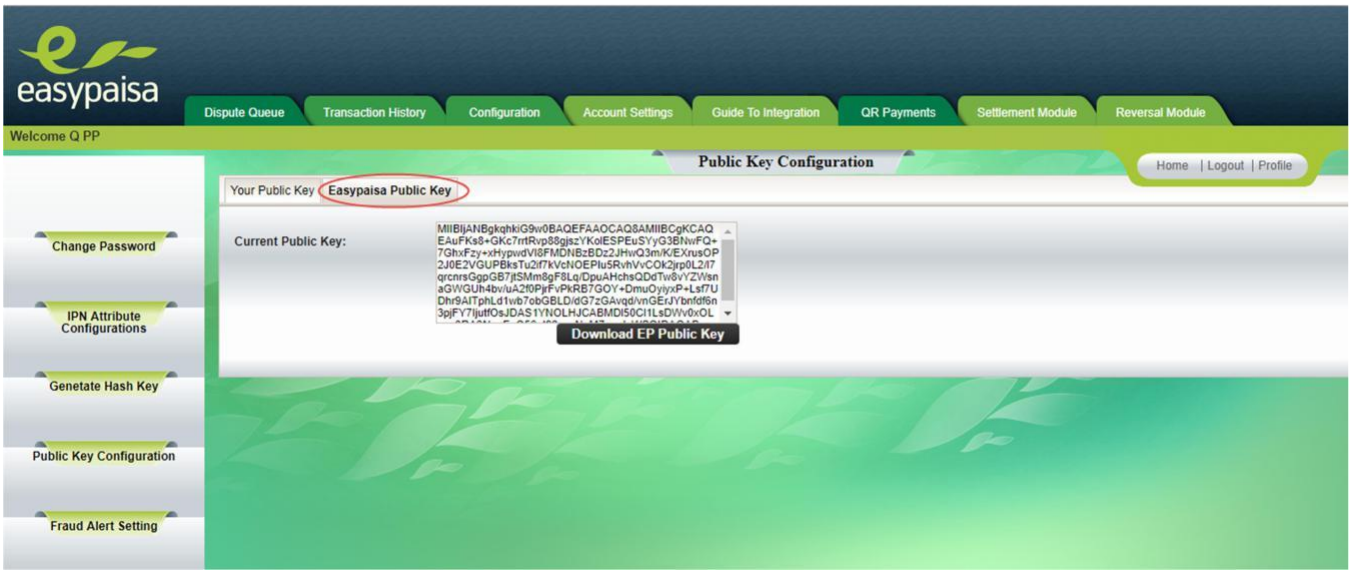

Figure 2 – Public Key Configuration (Download)

2.2 Signature Generation

The following figure illustrates a sample API request. The content enclosed by the two curly braces (inclusive) is what needs to be signed to create the RSA signature. And the generated signature is put in the signature parameter.

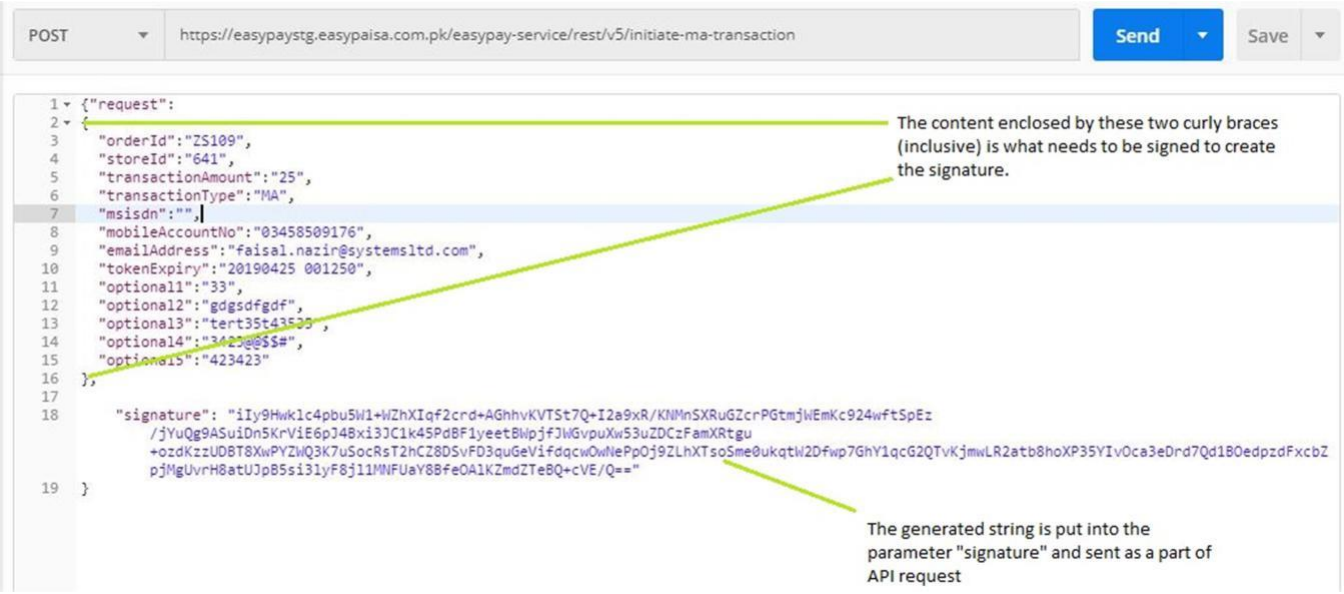

Figure 3 - Sample Request

1. **Extract the content to be signed and trim un-necessary spaces, For example:**


```
{"orderId":"ZS109","storeId":"641","transactionAmount":"25","transactionType":"MA","msisdn":"",
"mobileAccountNo":"03458509176","emailAddress":"faisal.nazir@systemsltd.com","tokenExpiry":"

20190425001250","optional1":"33","optional2":"gdgsdfgdf","optional3":"tert35t43535","optional4" :"3423@@$$#","optional5":"423423"}
```


2. **Hash the JSON content by using the SHA256withRSA algorithm. Then use the client's RSA private key to sign the value to obtain the signature. To achieve a better security level, the length of the RSA key/pairs must be 2048 bits. The following sample illustrates a raw signature:**


```
sXSGdA4tR4B0wnF6Nr8ShpfNMtHISWuzLpdKoNWskIgCcspMVFFzgkez4BrmtYXkMqZh1tyz/8sNNU3F2VTu0fO
x+ZP4CnmVjNu8bciu4hTtnytA3Y1GZ/yXAUD+mVYGXeyDO3fHrqDdI7k3xV/Q8FCOd0culTsN6RRMxMZq+oqR2x
+EjOXVofp7BZJwhQE8Us4Al573FQz5EHDvCLx35lq2hoMhXLCVLVDBHcpCgotuY4mg91MXh/qqcFGY/hQKhGdWA
bZ9tcp2Q5s2ESh37rqxnOwZrrepK8NC6vMBPUiSPxQefpev2nG0JzAHOjRPbBXYw1XBS6RBYKAmufZZiQ==
```


3. **Base64-encode the signature. A sample result is listed below:**


```
c1hTR2RBNHRSNEIwd25GNk5yOFNocGZOTXRISVNXdXpMcGRLb05Xc2tJZ0Njc3BNVkZGemdrZXo0QnJtdFlYa01
xWmgxdHl6LzhzTk5VM0YyVlR1MGZPeCtaUDRDbm1Wak51OGJjaXU0aFR0bnl0QTNZMUdaL3lYQVVEK21WWUdYZX
lETzNmSHJxRGRJN2szeFYvUThGQ09kMGN1bFRzTjZSUk14TVpxK29xUjJ4K0VqT1hWb2ZwN0JaSndoUUU4VXM0Q
Ww1NzNGUXo1RUhEdkNMeDM1bHEyaG9NaFhMQ1ZMVkRCSGNwQ2dvdHVZNG1nOTFNWGgvcXFjRkdZL2hRS2hHZFdB
YIo5dGNwMlE1czJFU2gzN3JxeG5Pd1pycmVwSzhOQzZ2TUJQVWlTUHhRZWZwZXYybkcwSnpBSE9qUlBiQlhZdzF
YQlM2UkJZS0FtdWZaWmlRPT0=
```


4. **Use the obtained string as the value of the Signature parameter.**

2.3 Signature Verification

A sample response is shown as below:


```
{
    "signature": "CdeHhpCnAvXfTCL8FQUIxWC4zre/OPEaYfThv49fp4vj+rqIri36Wf1PM0Jt1J1XwewXXrrxtCgOZme6ywZC3O3jIrvnqqgMsudSKijesdiKLfUraMWa97v
        /8t8NYsL13qgQq75DtVcCA/Kz66pwcz8x4nOr9MaqycNwNY928BH1vNUNM17ree/WIKn5sF+NGCi2NDQI4zpOkdDPdp4mkj/WTI
        /60dr7e3vzs1NtGOfebWLgAZWJ48uwdELuUeOuawFvQiZy+zQTFUyCmLerScrt2tz/StyR0esJnGwN2euzIf1WnCARMV9wTKJWtSlho8jYjDxAMKAnzpXu34Fe/w==",
    "response": {
        "orderId": "abc123",
        "storeId": 43,
        "transactionId": "253184",
        "transactionDateTime": "11/08/2018 11:30 PM",
        "responseCode": "0000",
        "responseDesc": "SUCCESS"
    }
}
```


The enclosed content (inclusive) is what needs to beverified against the signature using Easypay RSA public Key

After receiving a response, perform the following steps to verify the signature:

1. Split the full response contents to 2 parts, the response JSON string and the signature string, by using the regular expression instead of JSON object.
2. Trim any un-necessary spaces (if any) and hash the response JSON string by using the SHA256 algorithm to obtain a message digest.
3. Use the public key to decrypt the signature to a message digest.
4. Compare the two message digests obtained in step 2 and step 3. If the digests are the same, then it indicates that the signed data has not been changed.

3. Initiate MA Transaction API

The Initiate MA Transaction web-service is a web-service that can be used to initiate MA transaction in OPS. The parameters required by this API are mentioned below

3.1 Header

Header key	Header value	Description
Credentials	Base64Encoded username:password	Partner Account username & password for authentication

3.2 Request Parameters

Field Name	Description	Mandatory (M) / Optional (O)	Data Type
orderId	Merchant's system generated Order ID	M	String
storeId	Store ID It should be the same id which is associated with partner Account credentials sent in the header	M	Long
transactionAmount	Total transaction amount	M	String
transactionType	Type of transaction. Value : MA	M	String
mobileAccountNo	Format: 03xxxxxxxxx (11 - digits)	M	String

Field Name	Description	Mandatory (M) / Optional (O)	Data Type
emailAddress	Format : abc@xyz.com	M	String
optional1		O	String
optional2		O	String
optional3		O	String
optional4		O	String
optional5		O	String

3.3 Response Parameters

Field Name	Description	Data Type
orderId	Merchant's system generated Order ID	String
storeId	Store ID generated during merchant registration in Easypaisa	Long
transactionId	Transaction ID of Ericsson (EWP ID)	String
transactionDateTime	Format : dd/MM/yyyy hh:mm a	String
optional1		String
optional2		String
optional3		String
optional4		String
optional5		String
responseCode	Easypaisa generated response	String
responseDesc	Possible values are : Code : Description 0000 = SUCCESS 0001 = SYSTEM ERROR 0002 = REQUIRED FIELD MISSING 0005 = MERCHANT ACCOUNT NOT ACTIVE	String

Field Name	Description	Data Type
	0006 = INVALID STORE ID 0007 = STORE NOT ACTIVE 0008 = PAYMENT METHOD NOT ENABLED 0010 = INVALID CREDENTIALS 0013 = LOW BALANCE 0014 = ACCOUNT DOES NOT EXIST	